how many model layers are stored in VRAM. Current versions can take a number, `auto` or `all`, and automatic fitting is enabled by default.

If your supported GPU has enough VRAM, try:

```
llama cli -hf ggml-org/Qwen3-4B-GGUF:Q4_K_M
-c 4096 -ngl all
```

Then compare response speed with the default behaviour. If the model no longer fits, return to automatic selection or lower the number of GPU layers.

"GPU offload" is now a parameter you can change and observe rather than jargon from Chapter 4.

Step 5: Try CPU-only as an experiment

If your build and system permit it, set GPU layers to zero:

```
llama cli -hf ggml-org/Qwen3-4B-GGUF:Q4_K_M
-c 4096 -ngl 0
```

Run the baseline prompt again.

The result tells you what your actual computer does rather than what somebody else's benchmark predicts.

Step 6: Do not over-tune threads

The CLI lets you control CPU threads with -t, but its automatic defaults are a good place to begin. More threads do not guarantee proportionally higher speed, particularly once memory bandwidth becomes the bottleneck.

If you experiment, change the thread count in measured steps and rerun the same prompt. Stop when performance ceases to improve.

The golden rule remains: one change at a time.

Step 7: Optional – use the built-in web interface

llama.cpp can also start a local server with a web UI:

```
llama serve -hf ggml-org/Qwen3-4B-
GGUF:Q4_K_M
```

The server listens locally by default and gives you a browser interface for the model. It also exposes OpenAI-compatible endpoints, but we will deliberately leave API integration for the next Fast Track, where we start building applications.

Notice the architectural shift: the model can run as a local service while another interface talks to it.

Compare the three routes properly

You are not trying to crown a universal winner. You are deciding which experience suits you.

Run the same baseline prompt in each route with the same Qwen3-4B Q4_K_M model and a comparable context size. Record:

Setup friction: How long did it take to get from install to first answer?

First-response latency: How quickly did useful output begin?

Generation speed: Did the answer stream comfortably or feel sluggish?

Memory: How much RAM and VRAM did the session consume?

Control: Could you understand and change the settings you cared about?

Repeatability: How easily could you reproduce the exact configuration tomorrow?

Convenience: Which tool would you actually open every day?

LM Studio will often win on discoverability and a familiar graphical workflow. Ollama is attractive when you want model management and repeatable commands without living deep inside inference parameters. llama.cpp exposes more of the underlying mechanics and is invaluable when you want to understand or optimise how a GGUF is really running.

You do not have to choose only one. It is perfectly sensible to use LM Studio for exploration, Ollama for convenient recurring models and llama.cpp when testing performance or hardware behaviour.

A few chapters ago, a model name such as `Qwen3-4B-Q4_K_M.gguf` looked like alphabet soup. You can now identify what it is, find it, judge whether your hardware can handle it, download it, load it, choose how much of it reaches your GPU, change its context and make it answer locally.

More importantly, you have crossed the line between reading about local AI and operating it.

The next question is no longer "Can I run my own AI?"

It is "What can I actually do with it?"

YOUR FIRST LOCAL-AI SUCCESS CHECKLIST

Before moving on, make sure you can answer yes to these:

1. I can identify the exact model and quantisation I ran.
2. I know where that model came from and have checked its licence.
3. I can load it, ask a question and unload or stop it.
4. I can tell whether it is using CPU, GPU or a mixture where my runtime exposes that information.
5. I understand why increasing context can consume more memory.
6. I have successfully generated a response while disconnected from the internet using a setup intended for local inference.
7. I can delete a model I no longer need.
8. I have run one identical prompt across at least two runtimes and compared the experience.

That is a much bigger milestone than it looks.